

Expected Result

1. GET ทั้งหมด

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe http://localhost:8080/coffees  
[{"id":1,"name":"Espresso","price":45.0},{ "id":2,"name":"Latte","price":55.0}]
```

2. GET ตาม id

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe http://localhost:8080/coffees/1  
{"id":1,"name":"Espresso","price":45.0}
```

3. POST เพิ่มเมนูใหม่

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X POST http://localhost:8080/coffees `
>> -H "Content-Type: application/json" `
>> -d '{"name":"Cappuccino","price":60.0}'
{"id":3,"name":"Cappuccino","price":60.0}
```

4. PUT แก้ไข

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X PUT http://localhost:8080/coffees/2 `
>> -H "Content-Type: application/json" `
>> -d '{"name":"Latte","price":50.0}'
{"id":2,"name":"Latte","price":50.0}
```

5. DELETE ลบ

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X DELETE http://localhost:8080/coffees/3
```

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe http://localhost:8080/coffees  
[{"id":1,"name":"Espresso","price":45.0},{ "id":2,"name":"Latte","price":50.0}]
```

6. ค้นหาตามชื่อ

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe "http://localhost:8080/coffees/search?name=latte"  
[{"id":2,"name":"Latte","price":50.0}]
```

Discussion

1. HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

GET ใช้สำหรับดึงข้อมูล โดยไม่แก้ไขหรือเปลี่ยนแปลงข้อมูลที่มีอยู่ เช่น GET /coffees เพื่อดูรายการเมนูกาแฟทั้งหมด หรือ GET /coffees/{id} เพื่อดูเมนูตามรหัส

```
public List<Coffee> getAll() {  
    return coffees;  
}
```

POST ใช้สำหรับสร้างข้อมูลใหม่ เช่น POST /coffees เพื่อเพิ่มเมนูกาแฟใหม่เข้าสู่ระบบ

```
public Coffee create(Coffee newCoffee) {  
    Long newId = idCounter.incrementAndGet();  
    newCoffee.setId(newId);  
    coffees.add(newCoffee);  
    return newCoffee;  
}
```

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X POST http://localhost:8080/coffees `
>> -H "Content-Type: application/json" `
>> -d '{"name":"Cappuccino","price":60.0}'
{"id":3,"name":"Cappuccino","price":60.0}
```

PUT ใช้สำหรับแก้ไขหรืออัปเดตข้อมูลเดิมทั้งหมด เช่น PUT /coffees/{id} เพื่อแก้ไขชื่อหรือราคาของเมนูกาแฟ

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X PUT http://localhost:8080/coffees/2 `
>> -H "Content-Type: application/json" `
>> -d '{"name":"Latte","price":50.0}'
{"id":2,"name":"Latte","price":50.0}
```

DELETE ใช้สำหรับลบข้อมูล เช่น DELETE /coffees/{id} เพื่อลบเมนูกาแฟออกจากระบบ

```
public boolean delete(Long id) {  
    return coffees.removeIf(c -> c.getId().equals(id));  
}
```

```
PS D:\Principles3_1\Lab05_Spring_Been_Scope\coffeeshop> curl.exe -X DELETE http://localhost:8080/coffees/3
```

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมเมอร์

การแยก Controller และ Service ช่วยให้แต่ละส่วนมีหน้าที่ชัดเจน โดย Controller ทำหน้าที่รับคำขอ (HTTP Request) และส่งผลลัพธ์กลับ ส่วน Service ทำหน้าที่จัดการตรรกะการทำงานและข้อมูลของระบบ เมื่อโปรแกรมมีขนาดใหญ่ หากต้องการเปลี่ยนแปลงวิธีการจัดเก็บข้อมูล เช่น จาก List ไปเป็นฐานข้อมูลจริง ก็สามารถแก้ไขที่ Service ได้โดยไม่ต้องแก้ไข Controller ทำให้โค้ดดูแลรักษาง่าย และสามารถทดสอบแต่ละส่วนได้อย่างอิสระ

3. ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร (ตอบเป็นแนวคิดพอ)

ข้อมูลที่เก็บไว้ใน List ซึ่งอยู่ในหน่วยความจำ (Memory) จะหายไปเมื่อปิดโปรแกรมหรือรีสตาร์ทแอปพลิเคชัน เนื่องจากข้อมูลไม่ได้ถูกบันทึกลงในสื่อจัดเก็บถาวร หากต้องการให้ข้อมูลคงอยู่แม้ปิดโปรแกรม ควรเก็บข้อมูลไว้ในฐานข้อมูล เช่น H2, MySQL หรือ PostgreSQL และจัดการข้อมูลผ่าน Spring Data JPA

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

- **@RestController** ใช้กำหนดว่าคลาสนี้เป็น REST Controller ทำหน้าที่รับค่าขอจากผู้ใช้และส่งผลลัพธ์กลับในรูปแบบ JSON
- **@GetMapping** ใช้กำหนดเมธอดที่รองรับ HTTP GET สำหรับดึงข้อมูล
- **@PostMapping** ใช้กำหนดเมธอดที่รองรับ HTTP POST สำหรับเพิ่มข้อมูลใหม่
- **@PathVariable** ใช้รับค่าที่ส่งมาทาง URL เช่น id ใน /coffees/{id}
- **@RequestBody** ใช้รับข้อมูล JSON จาก Request Body และแปลงเป็น Java Object เพื่อให้สามารถนำไปใช้งานภายในโปรแกรมได้